



UNIVERSITÀ  
DEGLI STUDI  
DI PALERMO



# Introduzione a XML

CORSO DI  
PROGRAMMAZIONE WEB E MOBILE  
a.a. 2025/2026

*ING. LUCA CRUCIATA*

# Sommario

- Origini e storia di XML
- La sintassi di XML
- Document Object Model
- Document Type Definition (DTD)
- Alcuni esempi di marcatura

# Linguaggi di marcatura

- Un **linguaggio di marcatura** è un insieme di convenzioni per la marcatura di testi
- **Marcatura di documenti**
  - la marcatura (o etichettatura) permette di rendere **esplicita** un'interpretazione di un testo.
  - storicamente
    - annotazioni in un testo che descrivono al tipografo come stampare o comporre una parte del testo
  - oggi
    - **qualsiasi tipo di codice inserito in un testo in forma elettronica**

# Tipi di marcatura

- Due tipi di marcatura

- **marcatura procedurale**

- descrive come processare il documento
    - postscript, rtf, ecc.

"Hello world"



```
%!PS
/Courier findfont
20 scalefont
setfont
72 500 moveto
(Hello world!) show
showpage
```

Esempio postscript

- **marcatura descrittiva**

- descrive la **struttura logica** del documento
    - HTML, SGML, XML

# SGML - Standard Generalized Markup Language

- Il padre degli attuali linguaggi di marcatura
- È un **metalinguaggio di marcatura**, che permette di definire linguaggi di marcatura
  - estremamente espressivo e configurabile
  - l'alta espressività rende l'elaborazione automatica del testo complicata
  - utilizzato per grandi progetti di digitalizzazione del testo
  - non studiato espressamente per il Web

# SGML - Standard Generalized Markup Language

- Manca di alcune caratteristiche fondamentali per il Web:
  - gestione dei link
  - gestione del conflitto sui nomi delle etichette
  - tutti i documenti devono essere *validi* oltre a essere *ben formati*
- È troppo complicato per poter essere adoperato come linguaggio di marcatura

# HTML - HyperText Markup Language

- È un **linguaggio di marcatura** (non un metalinguaggio!)
  - definito in *direttamente in termini di SGML*
  - Insieme di etichette prefissato
- La marcatura non denota il “significato”, ovvero la struttura “logica” di un documento, *ma solo il suo formato in visualizzazione*

# HTML - HyperText Markup Language

- Studiato espressamente per il Web
  - collegamenti ipertestuali
  - immagini
  - marcatura finalizzata alla presentazione del documento come pagina Web
    - diversi tipi di titoli, tabelle, ecc.
- non c'è un legame tra marcatura e tipi di informazione rappresentati nel documento

# Una pagina HTML

```
<html>
  <head>
    <title>Prima pagina HTML</title>
  </head>

  <body>
    <p>Questa è la mia prima pagina HTML</p>
  </body>
</html>
```

# XML eXtensible Mark-up Language

- La marcatura è dettata dalla **struttura logica** del documento
- L'insieme di etichette può cambiare in base l'applicazione
- Fondamentale il concetto di tipo di documento
  - specificato attraverso una **Document Type Definition** o **DTD** (parte dello standard XML)
  - permette di dichiarare la struttura che tutti i documenti di un certo tipo devono rispettare

# XML - eXtensible Mark-up Language

- Naturale successore di HTML come linguaggio per il Web
  - più espressivo e flessibile, perché permette di definire tag personalizzati
  - la visualizzazione del documento è **indipendente dalla sua organizzazione logica**
    - per lo stesso documento XML è possibile definire più modalità di visualizzazione attraverso fogli di stile *XSL – XML Stylesheet Language*

# Esempio di documento XML

```
<libro>
  <intestazione>
    Le avventure di Pinocchio
  </intestazione>

  <autore>Carlo Collodi</autore>

  <titolo>Le avventure di Pinocchio</titolo>
  <capitolo>
    <intestazioneCapitolo>Capitolo I</intestazioneCapitolo>
    <titolo>Come andò che Maestro
      Ciliegia, falegname, trovò un pezzo di legno, che piangeva e rideva come un
      bambino.</titolo>
  </capitolo>
</libro>
```

# WHATWG

- Web Hypertext Application Technology Working Group
  - Comunità di sviluppatori che curano lo sviluppo delle tecnologie web
- Fondato da Apple, Mozilla Foundation, Opera Software nel 2004 in contrapposizione alla linea di sviluppo XML based adottata dal W3C
- Ha generato lo standard HTML5, recepito poi dal W3C

# Le origini di XML e HTML

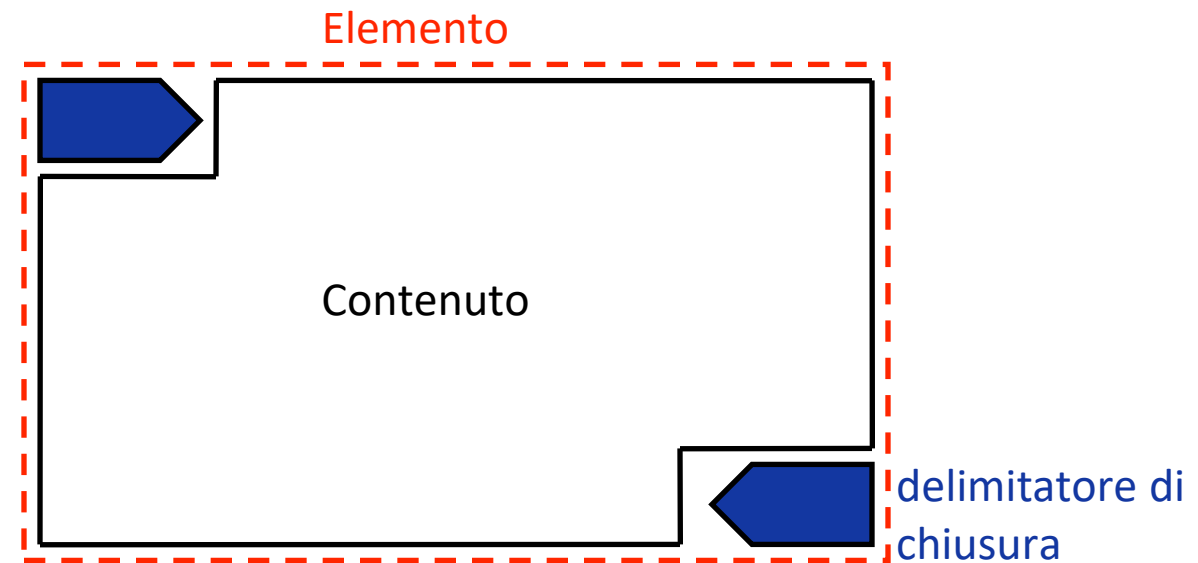
- 2012
  - Il WHATWG si separa dal W3C e inizia a sviluppare lo *HTML Living Standard*, di fatto il nuovo HTML5.
- 2014
  - *HTML 5*, snapshot del Living Standard, diventa una raccomandazione candidata W3C
- 2016
  - *HTML 5.1* diventa una raccomandazione candidata W3C

# I componenti della marcatura XML gli elementi

- Gli elementi rappresentano i blocchi costitutivi in cui si articola un testo
- Ogni elemento viene marcato in modo esplicito nel testo inserendo un delimitatore di apertura all'inizio dell'elemento e uno di chiusura alla fine

- Es: **<autore>**Carlo Collodi**</autore>**

delimitatore di  
apertura



# I componenti della marcatura XML

## i nomi degli elementi

- Ogni tipo di elemento è identificato da un **nome** (**etichetta** o **tag**)
  - il nome associato a ogni tipo di elemento è chiamato identificatore generico (generic identifier o GI)
- XML è **case-sensitive**
  - l'identificatore generico deve essere sempre specificato con lo stesso tipo di carattere, maiuscolo o minuscolo:  
`<tag>...</tag>`, `<TAG>...</TAG>`, `<Tag>...</Tag>`  
  
`<tag>...</TAG>` ***errato***

# I componenti della marcatura XML

## *i nomi degli elementi*

- Norme per la sintassi del nome degli elementi
  - possono contenere solo lettere, cifre, ., -, \_
  - possono iniziare solo con una lettera o con \_

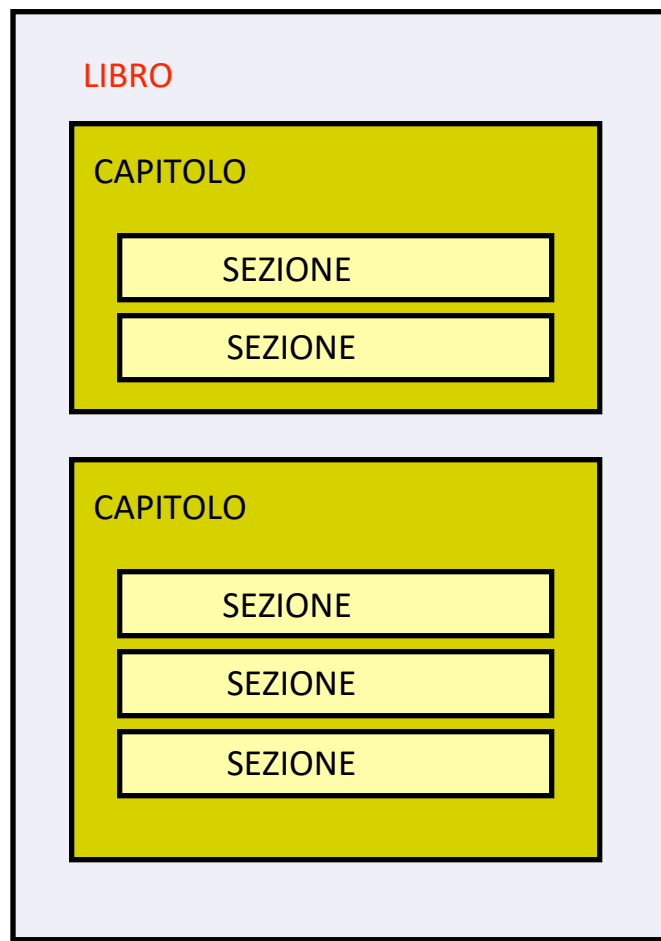
nomi consentiti: <autore.libro>, <\_autore>, <AUTORE-LIBRO>, <autore\_1>

nomi proibiti: <1autore>, <autore libro>, <autore;@?libro>

- non esiste un limite di lunghezza per il nome di un elemento

# I componenti della marcatura XML

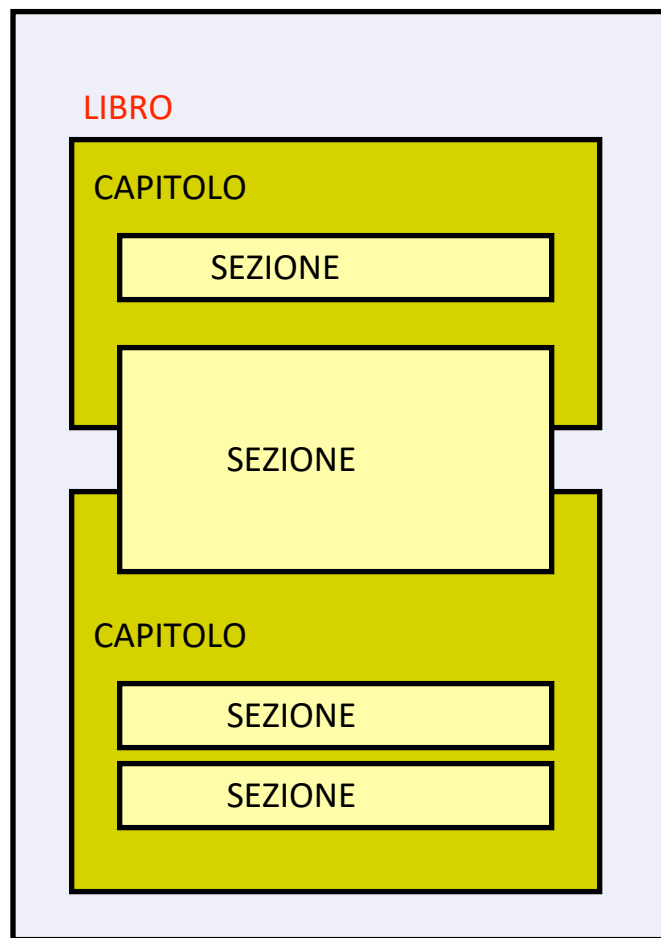
## *relazioni tra elementi*



- due elementi XML possono essere annidati l'uno nell'altro
- l'elemento più esterno è detto elemento genitore, quello interno elemento figlio
- questo meccanismo di annidamento degli elementi permette la rappresentazione di strutture gerarchiche di profondità variabile

# I componenti della marcatura XML

## *relazioni tra elementi*



- struttura XML mal formata in quanto esiste un elemento “a cavallo” di due elementi (annidamento improprio)
- in XML non è consentita la sovrapposizione tra elementi
- un elemento figlio deve essere completamente incluso nell’elemento padre

# I componenti della marcatura XML

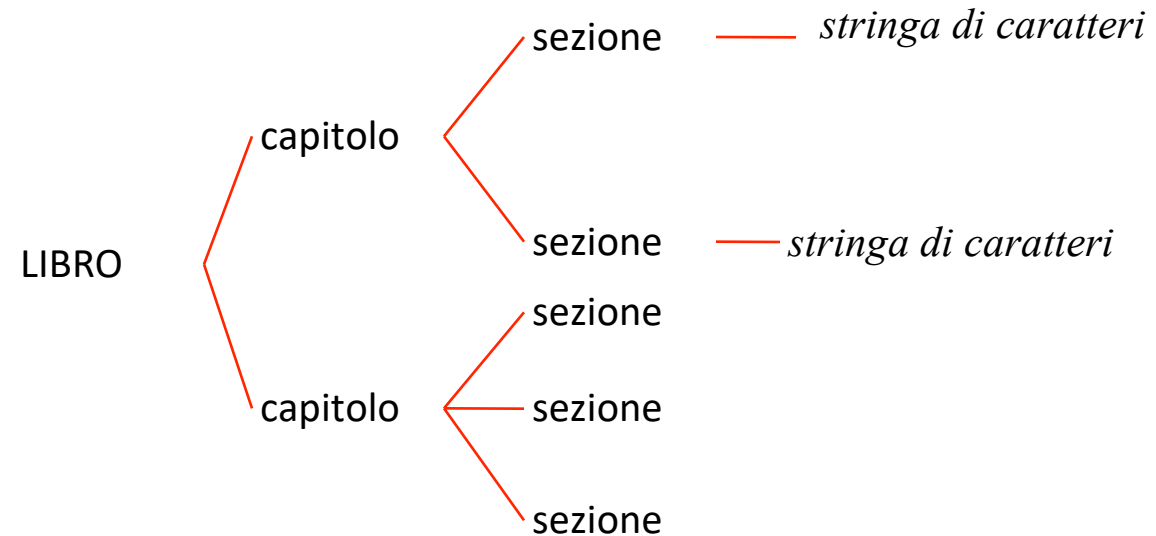
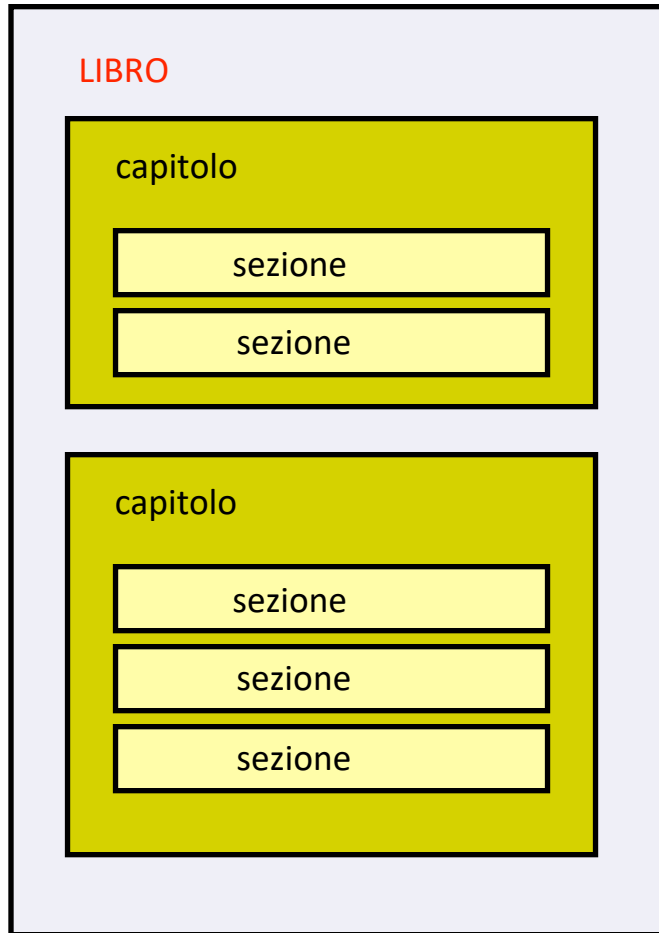
## *l'elemento radice*



- ogni documento XML ben formato deve contenere un elemento che contiene tutti gli altri elementi (elemento radice)
- la figura rappresenta una struttura mal formata in quanto in XML non è possibile avere più elementi a livello di radice
- ogni documento XML deve contenere uno e uno solo elemento radice

# I componenti della marcatura XML

## *il documento XML come albero*



# I componenti della marcatura XML

## *processing instructions*

- I dati contenuti in una istruzione di elaborazione vengono passate all'applicazione che usa il documento XML
- Solo delimitate dai caratteri `<?` e `?>`
- Sono formate da un *target* e da un *valore*

`<?xml-stylesheet type="text/xsl" href="usage.xsl"?>`

`<?xml-stylesheet type="text/css" href="mystyle.css"?>`

# I componenti della marcatura XML

## *processing instructions*

```
<?xml-stylesheet type="text/xsl" href="usage.xsl"?>
```

```
<?xml-stylesheet type="text/css" href="mystyle.css"?>
```

Con queste righe di codice stiamo importando due fogli di stile per il nostro file XML.

**NB** I file xsl non sono più supportati su diversi browser.

- Google Chrome e Edge non supportano più XSLT .
- Firefox e Safari supportano ancora XSLT

# I componenti della marcatura XML

## *processing instructions*

- Forniscono informazioni a strumenti che elaborano il file
- Non sono dati del documento stesso
- Non influiscono sulla struttura XML

`<?xml version="1.0" encoding="UTF-8"?>` (Specifica la versione XML e la codifica dei caratteri.)

`<?xml-stylesheet type="text/css" href="style.css"?>` (Indica un file CSS per formattare il documento XML.)

# I componenti della marcatura XML

## *Intestazione del documento*

- Tutti i documenti XML devono contenere una dichiarazione nel primo rigo

```
<?xml version="1.0" standalone="yes" ?>
```

```
<myMessage>
```

```
    <message>hello</message>
```

```
</myMessage>
```

# I componenti della marcatura XML

## *Intestazione del documento*

- Anche l'intestazione è un esempio di Processing Instruction
- Essa indica la versione di XML in cui è stato scritto il documento
- A volte indica anche se il documento è da ritenersi come una risorsa a sé stante o se legata ad altre

# I componenti della marcatura XML

## *contenuto di un elemento*

- Il contenuto di un elemento può essere costituito da:
- testo libero non contenente altri elementi (dati di tipo carattere)  
`<titolo>Le avventure di Pinocchio</titolo>`

- altri elementi (figli)

`<capitolo>`

`<titolo>Capitolo primo</titolo>`

`<capoverso>C'era una volta...</capoverso>`

`<capoverso>- Un re! - diranno subito i miei piccoli lettori.</capoverso>`

`</capitolo>`

# I componenti della marcatura XML

## *contenuto di un elemento*

- contenuto misto (elementi+ testo)

```
<titolo>Le avventure di Pinocchio  
  <sottotitolo>Storia di un burattino</sottotitolo>  
</titolo>
```

- Il contenuto di un elemento può essere “vuoto”

- due modi di denotare un elemento vuoto:

- coppia di delimitatori di apertura e chiusura

```
<salto_pagina></salto_pagina>
```

- etichetta di elemento vuoto

```
<salto_pagina/>
```

# I componenti della marcatura XML

## *gli attributi*

- Gli elementi XML possono essere dotati di uno o più attributi
  - gli attributi rappresentano informazioni aggiuntive che specificano alcune caratteristiche dell'elemento (ma che non fanno parte del contenuto del testo)

`nome_attributo= "valore"`

`<capoverso num="1">C'era una volta...</capoverso>`

# I componenti della marcatura XML

## *gli attributi*

- Nomi degli attributi
  - stesse restrizioni definite per i nomi degli elementi
- i valori degli attributi devono sempre essere racchiusi tra virgolette (singole o doppie)
  - nel caso in cui un valore contenga al suo interno delle virgolette, allora diventa obbligatorio differenziarle da quelle più esterne
- un attributo può ricorrere al massimo una volta all'interno di un elemento
  - ma ci possono essere più attributi differenti
- Gli attributi possono comparire solo nei tag di apertura degli elementi

# I componenti della marcatura XML

## *elementi vs. attributi*

```
<parola pos="nome"  
        num="sing"  
        gen="masc"  
        lemma="legno">  
    <orto>legno</orto>  
</parola>
```

```
<parola>  
    <orto>legno</orto>  
    <pos>nome</pos>  
    <accordo>  
        <num>sing</num>  
        <gen>masc</gen>  
    </accordo>  
</parola>
```

# I componenti della marcatura XML

## *elementi vs. attributi*

- Non è sempre facile stabilire quando preferire una codifica in termini di elementi o di attributi
- Spesso è una questione di “stile di codifica”
- Elementi e attributi hanno delle differenze espressive che possono o meno avere rilevanza nella definizione della nostra marcatura.

# I componenti della marcatura XML

## *elementi vs. attributi*

| Elementi                                                                           | Attributi                                                              |
|------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| Possono ricorrere <i>più volte</i> in un documento                                 | Ricorrono <i>al massimo una volta</i> in un elemento                   |
| E' possibile specificare <i>l'ordine</i> degli elementi nel documento              | Non è possibile stabilire l'ordine degli attributi                     |
| Un elemento può descrivere strutture complesse perché può contenere altri elementi | il valore di un attributo XML è semplicemente una stringa di caratteri |

# I componenti della marcatura XML

## *commenti*

- Ogni documento XML può contenere uno o più commenti
    - Sono ignorati dalle eventuali applicazioni che processino il documento
    - Possono apparire in qualunque punto all'interno del testo con le seguenti eccezioni:
      - Non possono apparire all'interno di un delimitatore di apertura o di chiusura di un elemento
      - Non possono apparire all'interno di un commento
- <!-- questo è un commento XML -->**

# I componenti della marcatura XML

## *namespace*

- Chiunque può definire i propri tag
  - Conflitti di nomi
  - Posso definire dei namespace
  - Il tag assume la forma **<prefissonamespace:tag>**

```
<image:images xmlns:image="urn:deitel:imageInfo">  
  <image:file filename="bunny.jpg" />  
</image:images>
```

# Caratteri e XML

- Tutti i file XML sono per default file di testo secondo la codifica Unicode UTF-8
  - indipendenza dei dati da piattaforme e totale interscambiabilità
  - è possibile specificare una codifica di caratteri diversa (nella Dichiarazione XML)

# Correttezza di un documento XML

- **Parsing** (o **analisi sintattica**)
  - il processo di analisi delle sequenze di token per determinarne la struttura grammaticale in relazione ad una data grammatica formale
  - Documento XML **ben formato** (well-formed) → sintatticamente corretto
- Parser non-validating → esegue **solo** il controllo sintattico per verificare se il documento è ben formato

# Correttezza di un documento XML

- La correttezza del documento riguarda anche l'uso dei corretti *nomi di tag e attributi* nonché delle corrette *relazioni di contenimento tra i tag* e dei *valori degli attributi*
- La specifica di questo tipo di correttezza si ottiene attraverso la **Document Type Definition (DTD)**
- Parser validating: leggono la DTD e stabiliscono se un documento XML è ad essa conforme o meno
  - Documento **valido**: sintatticamente corretto e conforme alle specifiche DTD

# Document Type Definition (DTD)

- Una Document Type Definition si può trovare nel prologo del documento XML

```
<!DOCTYPE prova [  
  <!ELEMENT miotag (#PCDATA) >  
>
```

- Gli elementi sono i componenti base di un documento XML
- PCDATA indica che il tag **miotag** racchiude del testo

# Document Type Definition (DTD)

- Una Document Type Definition può far riferimento a dichiarazioni esterne al documento XML

```
<!DOCTYPE prova SYSTEM "myDTD.dtd" [  
  <!ELEMENT miotag (#PCDATA) >  
]>
```

# Document Type Definition (DTD)

- Una Document Type Definition può far riferimento a dichiarazioni esterne al documento XML

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"  
"http://www.w3.org/TR/html4/strict.dtd">
```

Nome: *prefisso*//*proprietario*//*nome DTD*//*lingua*

*prefisso*: **ISO** (standard), + (approvato ISO), – (non approvato ISO)

URI di riferimento

# Document Type Definition (DTD)

```
<!ELEMENT persons (person) >
```

```
<!ELEMENT person (#PCDATA) >
```

In questo caso un tag **persons** contiene un tag **person**, che a sua volta contiene del testo

```
< persons>
```

```
  < person>John Smith</person>
```

```
</persons>
```

# Document Type Definition (DTD)

```
<!ELEMENT class (teacher, student) >  
<!ELEMENT teacher (#PCDATA) >  
<!ELEMENT student (#PCDATA) >
```

Il tag **class** contiene un tag **teacher** e un tag **student**. Questi due tag contengono testo

```
<class>  
  <teacher>prof</teacher>  
  <student>John</student>  
</class>
```

# Document Type Definition (DTD)

```
<!ELEMENT dessert (glace|fruit) >  
<!ELEMENT glace (#PCDATA) >  
<!ELEMENT fruit (#PCDATA) >
```

Il tag **dessert** contiene un tag **glace** o un tag **fruit**. Questi due tag contengono testo

```
<dessert>  
  <fruit>peach</fruit>  
</dessert>
```

oppure

```
<dessert>  
  <glace>chocolate</glace>  
</dessert>
```

# Document Type Definition (DTD)

```
<!ELEMENT compilation (song+) >
```

```
<!ELEMENT song (#PCDATA) >
```

Il tag `compilation` contiene **uno o più** tag `song`

```
<compilation>
```

```
  <song>Malo</song>
```

```
  <song>La Flaca</song>
```

```
  ...
```

```
</compilation>
```

# Document Type Definition (DTD)

```
<!ELEMENT compilation (song*) >  
<!ELEMENT song (#PCDATA) >
```

Il tag `compilation` contiene **zero o più** tag `song`

```
<compilation >  
  <song>Malo</song >  
  <song>La Flaca</song >
```

```
  ...  
</compilation>  
oppure  
<compilation/>
```

# Document Type Definition (DTD)

```
<!ELEMENT compilation (song?) >  
<!ELEMENT song (#PCDATA) >
```

Il tag `compilation` contiene **zero o un** tag `song`

```
<compilation >  
  <song>Malo</song >  
</compilation>
```

oppure

```
<compilation/>
```

# Document Type Definition (DTD)

- il contenuto di un elemento può essere di due tipo:
  - Vuoto (**EMPTY**)
  - Misto (**ANY**)
    - Solo PCDATA
    - solo altri elementi
    - PCDATA e altri elementi insieme

**<!ELEMENT vuoto EMPTY>**

**<vuoto/>**

# Document Type Definition (DTD)

- A ciascun tag possono essere associati degli attributi
  - Inizialmente dichiaro un elemento
  - Quindi definisco gli attributi x e y

```
<!ELEMENT posizione EMPTY>  
<!ATTLIST posizione x CDATA #REQUIRED>  
<!ATTLIST posizione y CDATA #REQUIRED>
```

- **#REQUIRED** indica che l'attributo è obbligatorio
- **CDATA** indica che il valore dell'attributo può contenere qualunque carattere

```
<posizione x="1" y="2" />
```

# Linguaggi di markup personalizzati

- **Mathematical Markup Language (MathML)**

- Sviluppato dal W3C per descrivere espressioni matematiche usando la sintassi XML

**<math>**

**<msqrt>**

**<msup>**

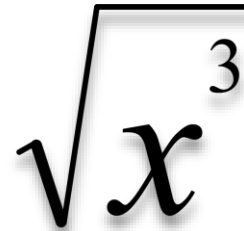
**<mi>x</mi>**

**<mn>3</mn>**

**</msup>**

**</msqrt>**

**</math>**


$$\sqrt{x^3}$$

# Linguaggi di markup personalizzati

- **eXtensible Business Reporting Language (XBRL)**
  - Permette di rappresentare dati di natura finanziaria, economica e amministrativa
- **Electronic Business Language (ebXML)**
  - Usato per lo scambio di informazioni commerciali e ed industriali
- **Commerce XML (cXML)**
  - Usato per descrivere dati di catalogo e svolgere transazioni elettroniche fra aziende che usano tali dati

# Linguaggi di markup personalizzati

- LegalXML

- Ideato per ridurre la ridondanza di informazioni e documenti giudiziari nei sistemi di gestione di tali dati

- NewsML

- Utilizzato nei sistemi di gestione delle news

- Rich Site Summary (RSS)

- Per creare canali che distribuiscono automaticamente le informazioni. RSS consente agli autori Web di creare un link che i visitatori possono selezionare per ricevere un determinato canale

# Linguaggi di markup personalizzati

- Scalable Vector Graphics (SVG)
  - Permette di descrivere immagini vettoriali

```
<svg width="300" height="300">  
  <circle style="fill:green;fill-opacity:0.5"  
    cx="50" cy="150" r="50"/>  
</svg>
```

- *Il DOM SVG è parte di HTML5*

# XSD (XML Schema Definition)

- Un file **XSD** (XML Schema Definition) definisce la struttura, i vincoli e i tipi di dato per i documenti XML, garantendone la validità. Agisce come un "progetto" o un insieme di regole che specificano quali elementi/attributi possono apparire, in che ordine e con quali formati (numeri, stringhe, date).
- `<xs:element>` definisce un **elemento XML** chiamato catalogo
- `<xs:sequence>` elemento è **complesso**, contiene altri elementi figli oppure ha attributi
- `<xs:complexType>` gli elementi figli devono apparire in quell'ordine

# Esempio...

<catalogo>

<libro>

<titolo>Il Signore degli Anelli</titolo>

<anno>1954</anno>

<prezzo>25.50</prezzo>

</libro>

</catalogo>

# Esempio DTD vs XSD

```
<!ELEMENT catalogo (libro+)>  
<!ELEMENT libro (titolo, anno,  
prezzo)>  
<!ELEMENT titolo (#PCDATA)>  
<!ELEMENT anno (#PCDATA)>  
<!ELEMENT prezzo (#PCDATA)>
```

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">  
  <xs:element name="catalogo">  
    <xs:complexType>  
      <xs:sequence>  
        <xs:element name="libro" maxOccurs="unbounded">  
          <xs:complexType>  
            <xs:sequence>  
              <xs:element name="titolo" type="xs:string"/>  
              <xs:element name="anno" type="xs:integer"/>  
              <xs:element name="prezzo" type="xs:decimal"/>  
            </xs:sequence>  
          </xs:complexType>  
        </xs:element>  
      </xs:sequence>  
    </xs:complexType>  
  </xs:element>  
</xs:schema>
```